

LAPORAN PRAKTIKUM STRUKTUR DATA
APLIKASI GUI SORTING (SHELLSORT, BUBBLESORT, MERGESORT
& QUICKSORT)



OLEH :

JOVANTRI IMMANUEL GULO

NIM 2411532014

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. Ir. Wahyudi, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 10 JUNI 2025

A. Pendahuluan

Sorting pada data merupakan salah satu proses penting dalam pengolahan data yang digunakan untuk menyusun elemen dalam urutan tertentu, baik secara ascending maupun descending. Dalam praktikum ini, dilakukan implementasi beberapa algoritma sorting, yaitu Shellsort, Bubblesort, Mergesort, dan Quicksort, yang masing-masing punya metode dan karakteristik yang berbeda. Shellsort merupakan pengembangan dari insertion sort dengan menggunakan gap untuk mengurangi jumlah pergeseran elemen. Bubblesort bekerja dengan cara membandingkan dan menukar elemen secara berulang hingga data terurut. Mergesort menggunakan metode pembagian berdasarkan jumlah elemen dan conquer untuk membagi dan menggabungkan elemen yang telah diurutkan. Sementara itu, Quicksort punya pivot sebagai inti dari pengurutan dan membagi elemen berdasarkan nilai pivot untuk melakukan pengurutan secara efisien.

B. Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Mampu memahami cara mengimplementasikan shell sort.
2. Mampu memahami cara mengimplementasikan bubble sort.
3. Mampu memahami cara mengimplementasikan merge sort.
4. Mampu memahami cara mengimplementasikan quick sort.

C. Langkah – langkah Pengerjaan

Berikut adalah langkah-langkah dalam pengerjaan praktikum kali ini:

1. Buat package baru dengan nama **pekan8**
2. Klik kanan pada package dan klik new, kemudian klik **Other**, kemudian klik **Window Builder**, klik **Swing Designer**, dan klik **JFrame**, kemudian buat nama file barunya dengan nama **ShellSortGUI**, tampilan GUI nya sama dengan InsertionSortGUI, hanya algoritma dan logika yang diubah pada class file
3. Import package, modul yang dibutuhkan, kemudian buat juga public class untuk file ShellSortGUI dengan extend dari JFrame serta **deklarasikan** juga beberapa variabel serta deklaratornya

```
1 package pekan8;
2
3 import java.awt.BorderLayout;
21
22 public class ShellSortGUI extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25     private int[] array;
26     private JLabel[] labelArray;
27     private JButton stepButton, resetButton, setButton;
28     private JTextField inputField;
29     private JPanel panelArray;
30     private JTextArea stepArea;
31
32     // Variabel state untuk visualisasi langkah per langkah
33     private int gap;
34     private int i; // Indeks luar (elemen yang akan disisipkan)
35     private int j; // Indeks dalam (posisi untuk penyisipan)
36     private int temp; // Nilai elemen yang sedang diproses
37     private boolean sorting = false;
38     private boolean isSwapping = false; // Flag penanda sedang dalam fase geser/sisip
39     private int stepCount = 1;
```

4. Untuk ShellSort, tampilannya sama dengan file **InsertionSortGUI** tapi berbeda pada bagian logikanya.
5. Pada bagian deklarasi ShellSort, kita perlu untuk mendeklarasikan
 - a. **int gap**, deklarasi ini nantinya akan digunakan untuk mendeteksi perbedaan kunci, shell sort ini membagi array menjadi beberapa bagian sub-array yang lebih kecil yang disebut sebagai gap
 - b. **int i**, deklarasi ini digunakan untuk mendeteksi indeks elemen saat ini pada sub-array yang tadinya sedang diurutkan, ini juga dipengaruhi oleh gap
 - c. **int j**, deklarasi ini digunakan untuk pergeseran pada suatu sub-array tetapi pergeserannya didasarkan pada gap

- d. **int temp**, deklarasi ini digunakan untuk menyimpan elemen sementara yang akan disisipkan, hal ini sama artinya seperti *key* pada InsertionSortGUI
- e. **private boolean isSwapping = false**, deklarasi ini digunakan untuk membantu melacak fase yang sedang berlangsung dalam method `performStep()`

```

32 // Variabel state untuk visualisasi langkah per langkah
33 private int gap;
34 private int i; // Indeks luar (elemen yang akan disisipkan)
35 private int j; // Indeks dalam (posisi untuk penyisipan)
36 private int temp; // Nilai elemen yang sedang diproses
37 private boolean isSorting = false;
38 private boolean isSwapping = false; // Flag penanda sedang dalam fase geser/sisip
39 private int stepCount = 1;

```

6. Pada bagian method **performStep()**, perbedaannya dengan `performStep` pada InsertionSortGUI adalah

```

87 private void performStep() {
88     if (!sorting) return;
89
90     // FIX: Logika reset highlight dipindahkan agar tidak menghapus highlight langkah sebelumnya terlalu cepat
91     resetHighlights();
92
93     // FASE 1: Sedang dalam proses menggeser dan mencari posisi sisip
94     if (isSwapping) {
95         // Cek apakah masih bisa geser ke kiri
96         if (j >= gap && array[j - gap] > temp) {
97             // Visualisasi perbandingan
98             labelArray[j - gap].setBackground(Color.CYAN); // Elemen pembanding
99             labelArray[j].setBackground(Color.YELLOW); // Posisi saat ini
100
101             // Lakukan pergeseran
102             array[j] = array[j - gap];
103             logStep("Geser " + array[j - gap] + " ke kanan (posisi " + j + ")");
104             updateLabels();
105
106             j -= gap; // Mundur sejauh gap untuk perbandingan selanjutnya
107         } else {
108             // Posisi sudah ditemukan, sisipkan elemen
109             array[j] = temp;
110             logStep("Sisipkan " + temp + " di posisi " + j);
111             labelArray[j].setBackground(Color.RED); // Tandai posisi penyisipan
112             updateLabels();
113
114             // Sian untuk elemen berikutnya di iterasi i
115             i++;
116             isSwapping = false; // Keluar dari fase penyisipan
117         }
118     }
119     // FASE 2: Memulai proses untuk elemen baru (i) atau mengganti gap
120     else {
121         // Jika iterasi i untuk gap saat ini sudah selesai
122         if (i >= array.length) {
123             gap /= 2; // Perkecil gap
124             if (gap > 0) {
125                 logStep("PASS SELESAI. Ganti gap menjadi " + gap);
126                 i = gap; // Mulai iterasi i dari gap yang baru
127             } else {
128                 // Sorting selesai total
129                 logStep("SORTING SELESAI!");
130                 sorting = false;
131                 stepButton.setEnabled(false);
132                 for (JLabel label : labelArray) {
133                     label.setBackground(Color.LIGHT_GRAY);
134                 }
135                 JOptionPane.showMessageDialog(this, "Shell Sort selesai!");
136                 return;
137             }
138         }
139
140         // Memulai fase penyisipan untuk elemen di posisi i
141         temp = array[i];
142         j = i;
143         logStep("Ambil elemen ke-" + i + " (nilai: " + temp + ") untuk disisipkan");
144         labelArray[i].setBackground(Color.YELLOW); // Tandai elemen yang akan disisipkan
145         isSwapping = true;
146     }
147 }

```

- a. Dibagi menjadi 2 fase menggunakan flag `swapping`
 - b. Menambahkan method `reset highlight` pada langkah awalnya
 - c. Mengimplementasikan logika `multiple pass` dengan jarak atau `gap`
7. Pada method **setArrayFromInput**, perbedaannya adalah

```

156 private void setArrayFromInput() {
157     String text = inputField.getText().trim();
158     if (text.isEmpty()) {
159         JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!", "Error", JOptionPane.ERROR_MESSAGE);
160         return;
161     }
162     String[] parts = text.split(",");
163     array = new int[parts.length];
164
165     try {
166         for (int k = 0; k < parts.length; k++) {
167             array[k] = Integer.parseInt(parts[k].trim());
168         }
169     } catch (NumberFormatException e) {
170         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
171         return;
172     }
173
174     // Reset state
175     gap = array.length / 2;
176     i = gap;
177     j = i;
178     stepCount = 1;
179     isSwapping = false;
180
181     stepArea.setText("Array Awal: " + Arrays.toString(array) + "\n");
182     logStep("Mulai dengan gap = " + gap);
183
184     sorting = true;
185     stepButton.setEnabled(true);
186     panelArray.removeAll();
187     labelArray = new JLabel[array.length];
188
189     for (int k = 0; k < array.length; k++) {
190         labelArray[k] = new JLabel(String.valueOf(array[k]));
191         labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
192         labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
193         labelArray[k].setPreferredSize(new Dimension(50, 50));
194         labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
195         // FIX: Menghapus karakter spasi aneh
196         labelArray[k].setOpaque(true);
197         labelArray[k].setBackground(Color.WHITE);
198         panelArray.add(labelArray[k]);
199     }
200     panelArray.revalidate();
201     panelArray.repaint();
202 }

```

- a. Penambahan inisialisasi variabel gap
 - b. Penambahan start index atau index awal (i) yang diatur berdasarkan gap atau jarak, bukan nilai tetap 1
 - c. Penambahan logging khusus untuk gap
8. Pada method **UpdateLabels**, perbedaannya adalah

```

204 private void updateLabels() {
205     for (int k = 0; k < array.length; k++) {
206         labelArray[k].setText(String.valueOf(array[k]));
207     }
208 }

```

- a. Melakukan pengupdatean pada setText saja, tidak mengatur warna
9. Pada method **reset**, perbedaannya adalah

```

217 private void reset() {
218     inputField.setText("");
219     panelArray.removeAll();
220     panelArray.revalidate();
221     panelArray.repaint();
222     stepArea.setText("");
223     stepButton.setEnabled(false);
224     sorting = false;
225     array = null;
226     labelArray = null;
227 }

```

- a. Reset variabel khusus Shell Sort (gap, isSwapping)
 - b. Penanganan null safety untuk labelArray
10. Method **logStep**

```

149 private void logStep(String message) {
150     stepArea.append("Langkah " + stepCount + ": " + message + "\n");
151     stepArea.append(" Array: " + Arrays.toString(array) + "\n\n");
152     stepArea.setCaretPosition(stepArea.getDocument().getLength());
153     stepCount++;
154 }

```

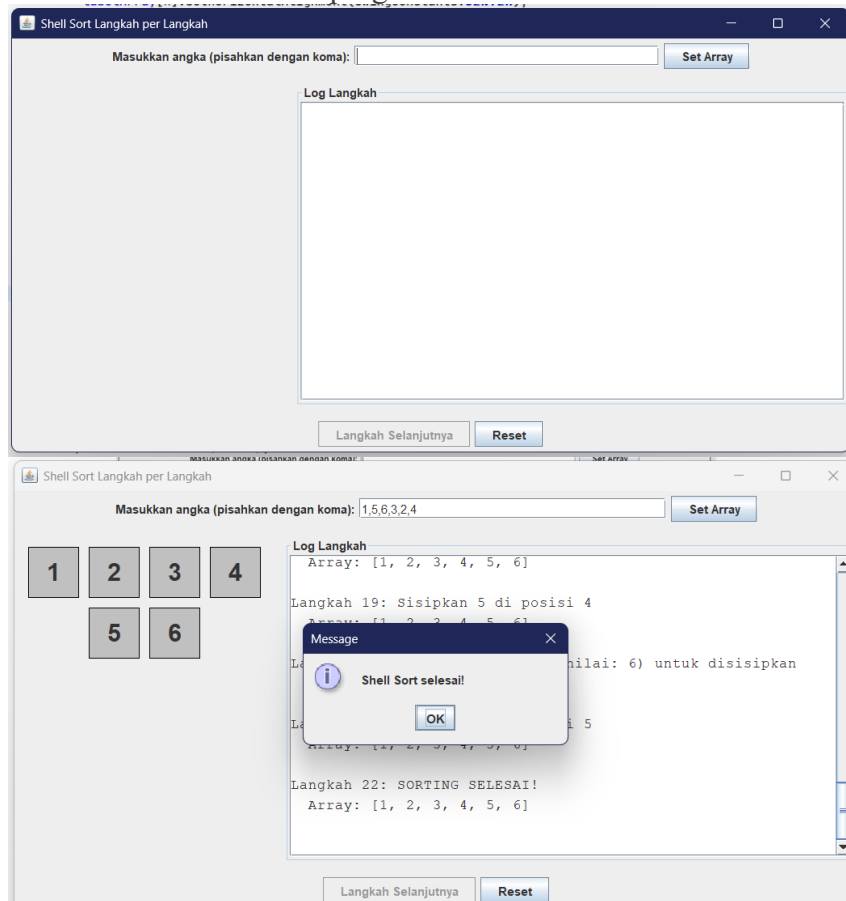
- a. Standarisasi format logging dengan menampilkan logging tiap stepnya

11. Method **ResetHighlights**

```
210 private void resetHighlights() {  
211     if (labelArray == null) return;  
212     for (JLabel label : labelArray) {  
213         label.setBackground(Color.WHITE);  
214     }  
215 }
```

- a. Mereset warna label sebelum langkah selanjutnya dengan `label.setBackground`

12. Berikut adalah hasil dari program



13. Untuk **BubbleSort**, sama dengan InsertionSort, hanya terdapat beberapa perbedaan pada method dan deklarasi.

14. Pada bagian deklarasi, perbedaannya adalah

```
33 private int i = 0, j = 0;  
34 private boolean sorting = false;  
35 private int stepCount = 1;
```

- a. Menggunakan dua counter karena algoritmanya bekerja dalam dua lapis iterasi (pass dan perbandingan yang terdapat pada pass)

15. Pada method **PerformStep**, perbedaannya adalah


```

101 private void performStep() {
102     if (i ≥ array.length - 1 || !sorting) {
103         sorting = false;
104         stepButton.setEnabled(false);
105         JOptionPane.showMessageDialog(this, "Sorting selesai!");
106         for (JLabel label : labelArray) {
107             label.setBackground(Color.LIGHT_GRAY);
108         }
109         return;
110     }
111
112     resetHighlights();
113
114     StringBuilder stepLog = new StringBuilder();
115
116     labelArray[j].setBackground(Color.CYAN);
117     labelArray[j + 1].setBackground(Color.CYAN);
118
119     stepLog.append("Langkah ").append(stepCount).append(": Membandingkan arr[").append(j).append("] dan arr[").append(j + 1).append("].\n");
120
121     if (array[j] > array[j + 1]) {
122         int val1 = array[j];
123         int val2 = array[j + 1];
124
125         int temp = array[j];
126         array[j] = array[j + 1];
127         array[j + 1] = temp;
128
129         labelArray[j].setBackground(Color.RED);
130         labelArray[j + 1].setBackground(Color.RED);
131         stepLog.append(" → Tukar! Karena ").append(val1).append(" > ").append(val2).append(").\n");
132     } else {
133         stepLog.append(" → Tidak ada pertukaran.\n");
134     }
135
136     updateLabels();
137     stepLog.append(" Hasil: ").append(arrayToString(array)).append("\n\n");
138
139     stepArea.append(stepLog.toString());
140     stepArea.setCaretPosition(stepArea.getDocument().getLength());
141     j++;
142
143     if (j ≥ array.length - i - 1) {
144         labelArray[array.length - i - 1].setBackground(Color.LIGHT_GRAY);
145         j = 0;
146         i++;
147     }
148
149     stepCount++;
150
151     if (i ≥ array.length - 1) {
152         sorting = false;
153         stepButton.setEnabled(false);
154         JOptionPane.showMessageDialog(this, "Sorting selesai!");
155         for (JLabel label : labelArray) {
156             label.setBackground(Color.LIGHT_GRAY);
157         }
158     }
159 }
160 }

```

- Mengecek if ($i \geq \text{array.length} - 1 \parallel !\text{sorting}$) di awal untuk kondisi selesai atau belum dimulai.
- Mereset highlight: `resetHighlights()`;
- Membandingkan `array[j]` dan `array[j+1]`.
- Melakukan pertukaran (swap) jika `array[j] > array[j+1]`.
- Menginkrementasi `j` (`j++`).
- Mereset `j` ke 0 dan menginkrementasi `i` (`i++`) ketika satu pass (`j ≥ array.length - i - 1`) selesai.
- Mengimplementasikan algoritma Bubble Sort, secara berulang membandingkan pasangan elemen yang berdekatan dan menukarnya jika urutannya salah, sehingga elemen terbesar "menggelembung" ke posisi akhirnya di setiap pass

16. Pada method **setArrayFromInput**, perbedaannya adalah

```

162= private void setArrayFromInput() {
163     String text = inputField.getText().trim();
164     if (text.isEmpty()) {
165         JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!", "Error", JOptionPane.ERROR_MESSAGE);
166         return;
167     }
168     String[] parts = text.split(",");
169     if (parts.length == 0 || (parts.length == 1 && parts[0].trim().isEmpty())) {
170         JOptionPane.showMessageDialog(this, "Masukkan setidaknya satu angka!", "Error", JOptionPane.ERROR_MESSAGE);
171         return;
172     }
173     array = new int[parts.length];
174
175     try {
176         for (int k = 0; k < parts.length; k++) {
177             array[k] = Integer.parseInt(parts[k].trim());
178         }
179     } catch (NumberFormatException e) {
180         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
181         return;
182     }
183
184     i = 0;
185     j = 0;
186     stepCount = 1;
187     sorting = true;
188     stepButton.setEnabled(true);
189     stepArea.setText("Array Awal: " + arrayToString(array) + "\n\n");
190
191     panelArray.removeAll();
192     JLabel[] labelArray = new JLabel[array.length];
193     for (int k = 0; k < array.length; k++) {
194         labelArray[k] = new JLabel(String.valueOf(array[k]));
195         labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
196         labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
197         labelArray[k].setPreferredSize(new Dimension(50, 50));
198         labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
199         labelArray[k].setOpaque(true);
200         labelArray[k].setBackgroundColor(Color.WHITE);
201         panelArray.add(labelArray[k]);
202     }
203
204     panelArray.revalidate();
205     panelArray.repaint();
206 }

```

- a. Kedua variabel i dan j disetel ke 0.
- b. Semua elemen di awal akan diberi warna WHITE (putih).
- c. Bedanya terletak pada bagian memulai perbandingan dari awal, tidak ada elemen yang langsung ditandai sebagai "terurut". Semua elemen diberi warna dasar putih

17. Pada method **updateLabels** dan method **resetHighlights**, sama saja dengan method pada sort sebelumnya, yaitu pada ShellSort

18. Pada method **reset**, perbedaannya adalah

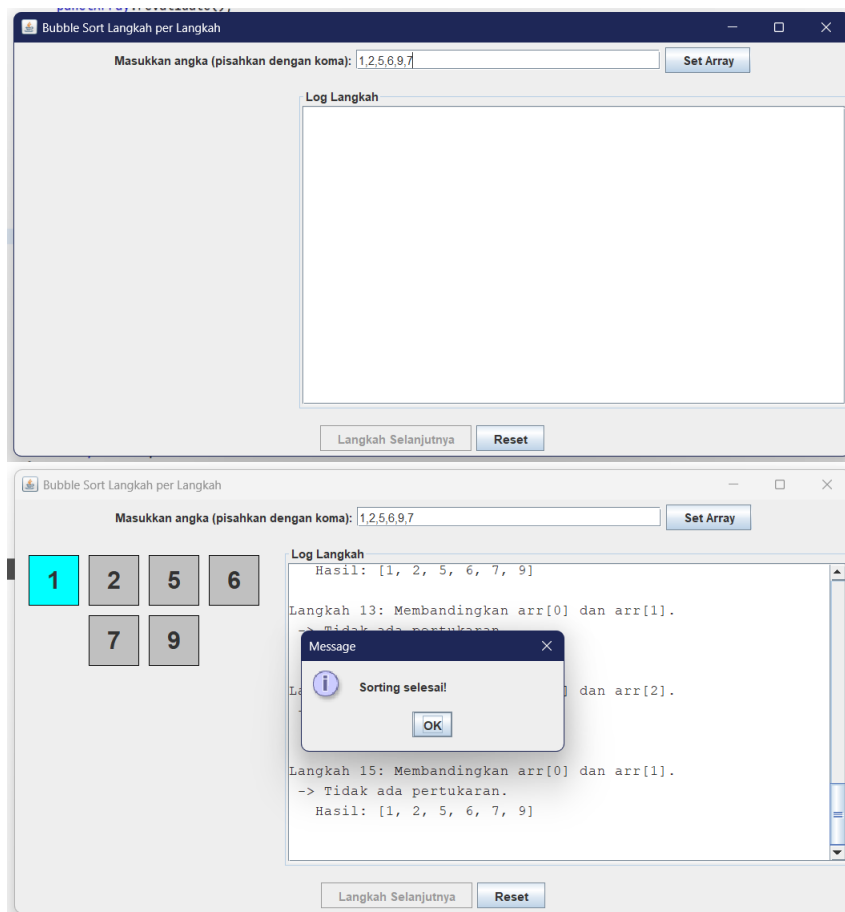
```

222= private void reset() {
223     inputField.setText("");
224     panelArray.removeAll();
225     panelArray.revalidate();
226     panelArray.repaint();
227     stepArea.setText("");
228     stepButton.setEnabled(false);
229     sorting = false;
230     i = 0;
231     j = 0;
232     stepCount = 1;
233     array = null;
234 }

```

- a. Variabel array disetel ke null.
- b. Variabel labelArray tidak secara eksplisit disetel ke null. Meskipun panelArray.removeAll() akan menghapus semua komponen label dari panel tampilan.
- c. Perbedaan: Referensi labelArray tidak dihapus secara eksplisit.

19. Berikut adalah hasilnya



20. Untuk **QuickSort**, tampilannya sama dengan **InsertionSort**, hanya terdapat beberapa perbedaan pada algoritma logikanya.

21. Pada pendeklarasian, terdapat perbedaan yaitu

```

32     private int i = 0, j = 0;
33     private boolean sorting = false;
34     private int stepCount = 1;
35
36     private Stack<int[]> stack;
37     private int low, high, pivot;
38     private boolean partitioning = false;

```

- a. Penambahan variabel `stack`, `low`, `high`, `pivot`, dan `partitioning`, serta perbedaan inisialisasi dan penggunaan `i` dan `j`

22. Pada method **performStep**, perbedaannya adalah


```

101 private void performStep() {
102
103     if (stack.isEmpty() && !partitioning) {
104         if (sorting) {
105             sorting = false;
106             stepButton.setEnabled(false);
107             resetHighlights(true); // Warnai semua jadi abu-abu
108             stepArea.append("\nQuick Sort selesai.\n");
109             JOptionPane.showMessageDialog(this, "Quick Sort selesai!");
110         }
111         return;
112     }
113
114     resetHighlights(false);
115
116     if (!partitioning) {
117         int[] range = stack.pop();
118         low = range[0];
119         high = range[1];
120
121         if (low >= high) {
122             performStep();
123             return;
124         }
125
126         pivot = array[high];
127         i = low - 1;
128         j = low;
129         partitioning = true;
130         stepArea.append("--- Mulai Partisi ---\n");
131         logStep("Range: [" + low + " .. " + high + "], Pivot: " + pivot + " (di indeks " + high + ")");
132         highlightPivot(high);
133         return;
134     }
135
136     if (j < high) {
137         highlightCompare(j, high);
138         if (array[j] <= pivot) {
139             i++;
140             logStep("Cek " + array[j] + " <= " + pivot + ". Benar. Pindahkan ke bagian kiri.");
141             if (i != j) {
142                 logStep(" → Tukar arr[" + i + "]=" + array[i] + " dengan arr[" + j + "]=" + array[j]);
143                 swap(i, j);
144                 updateLabels();
145             }
146         } else {
147             logStep("Cek " + array[j] + " <= " + pivot + ". Salah. Lewatkan.");
148         }
149         j++;
150         return;
151     }
152
153     int pivotFinalIndex = i + 1;
154     logStep("Partisi selesai. Pindahkan pivot " + pivot + " ke posisi finalnya di indeks " + pivotFinalIndex);
155     swap(pivotFinalIndex, high);
156     updateLabels();
157
158     partitioning = false;
159
160     labelArray[pivotFinalIndex].setBackground(Color.LIGHT_GRAY);
161
162     stepArea.append("--- Selesai Partisi ---\n\n");
163
164     stack.push(new int[] {low, pivotFinalIndex - 1});
165     stack.push(new int[] {pivotFinalIndex + 1, high});

```

- a. performStep() di Quick Sort mengelola seluruh siklus partisi untuk satu sub-array, termasuk memilih pivot, memindahkan elemen, dan menempatkan pivot di posisi finalnya. Ini sangat berbeda dari logika pergeseran satu elemen di Insertion Sort.
- b. Program memilih pivot (elemen terakhir dari sub-array), mengatur i dan j untuk proses partisi, dan mulai menandai bahwa program sedang dalam mode partisi.
- c. Selama partisi (j kurang dari high): Program membandingkan elemen saat ini (array[j]) dengan pivot. Jika array[j] lebih kecil atau sama dengan pivot, i akan bertambah, dan array[j] akan ditukar dengan array[i] (memastikan elemen yang lebih kecil dari pivot berada di sisi kiri). Jika tidak, elemen dilewatkan. j selalu bertambah.
- d. Setelah j mencapai high, proses partisi selesai. Pivot akan ditukar ke posisi akhirnya (i+1), yang membagi sub-array menjadi dua bagian (elemen lebih kecil dari pivot dan elemen lebih besar dari pivot).
- e. Status partitioning disetel ulang menjadi false.
- f. Dua sub-array baru (di sebelah kiri dan kanan pivot) kemudian didorong ke stack untuk diproses selanjutnya.
- g. Visualisasi menggunakan warna oranye untuk pivot, biru muda untuk elemen yang sedang dibandingkan, dan abu-abu terang untuk elemen yang sudah berada di posisi terurut akhirnya (setelah dipartisi).

23. Pada method **setArrayFromInput**, terdapat perbedaan yaitu

```

179= private void setArrayFromInput() {
180     String text = inputField.getText().trim();
181     if (text.isEmpty()) {
182         JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!", "Error", JOptionPane.ERROR_MESSAGE);
183         return;
184     }
185     String[] parts = text.split(",");
186     if (parts.length < 2) {
187         JOptionPane.showMessageDialog(this, "Masukkan setidaknya dua angka!", "Error", JOptionPane.ERROR_MESSAGE);
188         return;
189     }
190     array = new int[parts.length];
191
192     try {
193         for (int k = 0; k < parts.length; k++) {
194             array[k] = Integer.parseInt(parts[k].trim());
195         }
196     } catch (NumberFormatException e) {
197         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
198         return;
199     }
200     labelArray = new JLabel[array.length];
201     panelArray.removeAll();
202
203     for (int k = 0; k < array.length; k++) {
204         labelArray[k] = new JLabel(String.valueOf(array[k]));
205         labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
206         labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
207         labelArray[k].setPreferredSize(new Dimension(50, 50));
208         labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
209         labelArray[k].setOpaque(true);
210         labelArray[k].setBackgroundColor(Color.WHITE);
211         panelArray.add(labelArray[k]);
212     }
213
214     stack.clear();
215     stack.push(new int[] {0, array.length - 1});
216     sorting = true;
217     partitioning = false;
218     stepCount = 1;
219     stepArea.setText("Array Awal: " + java.util.Arrays.toString(array) + "\n\n");
220     stepButton.setEnabled(true);
221
222     panelArray.revalidate();
223     panelArray.repaint();
224 }

```

- Setelah pengguna memasukkan angka, semua elemen visual diatur ulang ke warna putih, dan sebuah Stack baru diisi dengan rentang indeks seluruh daftar awal (dari 0 hingga elemen terakhir) agar Quick Sort bisa mulai bekerja.
- Terdapat juga pemeriksaan tambahan untuk memastikan pengguna memasukkan setidaknya dua angka
- Persiapan ini sangat penting bagi Quick Sort karena ia mengandalkan Stack untuk mengelola seluruh proses pengurutan sejak awal.

24. Method **highlightPivot**, digunakan untuk memberi warna kuning pada elemen yang menjadi pivot (patokan)

```

168= private void highlightPivot(int index) {
169     labelArray[index].setBackgroundColor(Color.ORANGE);
170 }

```

25. Method **highlightCompare**, menyorot elemen yang sedang dibandingkan dengan warna biru muda dan juga memberi warna kuning pada elemen pivot

```

172= private void highlightCompare(int jIndex, int pivotIndex) {
173     labelArray[jIndex].setBackgroundColor(Color.CYAN);
174     if (labelArray[pivotIndex].getBackground() != Color.LIGHT_GRAY) {
175         labelArray[pivotIndex].setBackgroundColor(Color.ORANGE);
176     }
177 }

```

26. Method **resetHighlights**, mengatur ulang warna latar belakang semua label menjadi putih

```

232= private void resetHighlights(boolean markAsSorted) {
233     if (labelArray == null) return;
234     for (JLabel label : labelArray) {
235         if (markAsSorted || label.getBackground() == Color.LIGHT_GRAY) {
236             label.setBackgroundColor(Color.LIGHT_GRAY);
237         } else {
238             label.setBackgroundColor(Color.WHITE);
239         }
240     }
241 }

```

27. Method **swap**, untuk menukar posisi dua elemen dalam daftar

```

243 private void swap(int a, int b) {
244     int temp = array[a];
245     array[a] = array[b];
246     array[b] = temp;
247 }

```

28. Method **updateLabels**, fokus pada memperbarui teks angka di setiap label

```

226 private void updateLabels() {
227     for (int k = 0; k < array.length; k++) {
228         labelArray[k].setText(String.valueOf(array[k]));
229     }
230 }

```

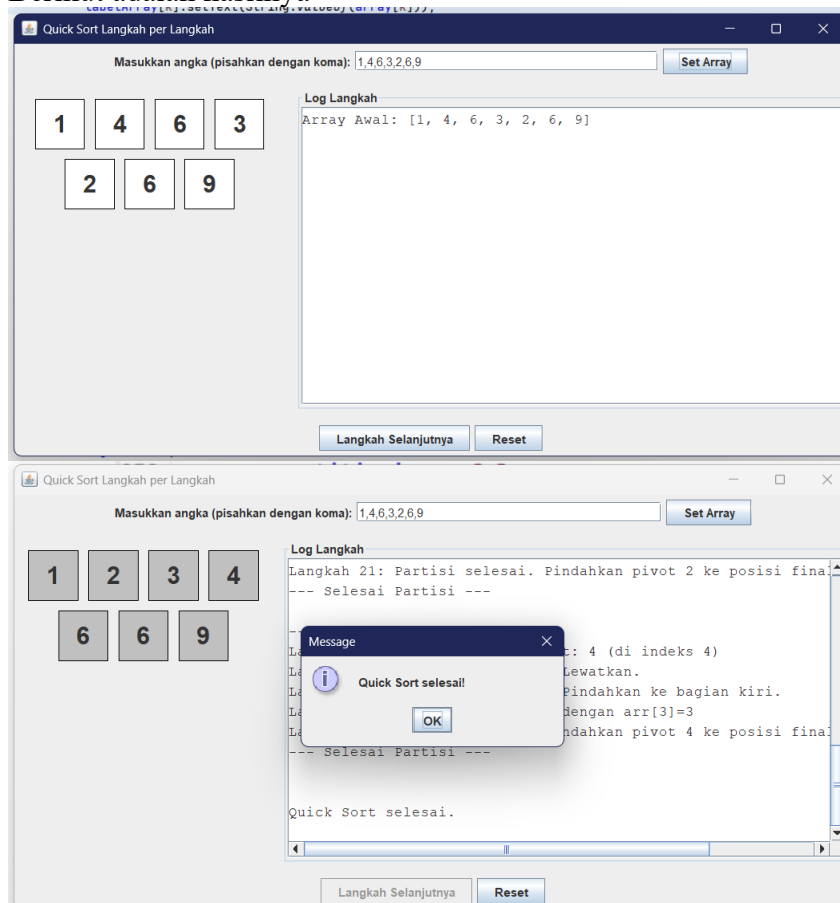
29. Method **reset**, selain mengosongkan input dan tampilan, metode reset di QuickSortGUI secara spesifik mengosongkan stack (yang tidak ada di InsertionSortGUI) dan mengatur ulang variabel sorting, partitioning, serta stepCount ke nilai awal

```

249 private void reset() {
250     inputField.setText("");
251     panelArray.removeAll();
252     panelArray.revalidate();
253     panelArray.repaint();
254     stepArea.setText("");
255     stepButton.setEnabled(false);
256     if(stack != null) stack.clear();
257     sorting = false;
258     partitioning = false;
259     stepCount = 1;
260     array = null;
261     labelArray = null;
262 }

```

30. Berikut adalah hasilnya



31. Untuk **MergeSort**, sama dengan InsertionSort, hanya beberapa berbeda pada algoritma pengurutannya.
32. Pada method **setArrayFromInput**, mergeQueue dikosongkan, lalu generateMergeSteps(0, array.length - 1) dipanggil untuk menyiapkan urutan langkah penggabungan. Variabel isMerging disetel ke false (belum mulai penggabungan sebenarnya), dan tombol 'Langkah Selanjutnya' diaktifkan

```

183= private void setArrayFromInput() {
184     String text = inputField.getText().trim();
185     if (text.isEmpty()) {
186         JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!", "Error", JOptionPane.ERROR_MESSAGE);
187         return;
188     }
189     String[] parts = text.split(",");
190     array = new int[parts.length];
191     try {
192         for (int i = 0; i < parts.length; i++) {
193             array[i] = Integer.parseInt(parts[i].trim());
194         }
195     } catch (NumberFormatException e) {
196         JOptionPane.showMessageDialog(this, "Masukkan hanya angka!", "Error", JOptionPane.ERROR_MESSAGE);
197         return;
198     }
199
200     stepCount = 1;
201     isMerging = false;
202     copying = false;
203     if (mergeQueue != null) {
204         mergeQueue.clear();
205     }
206
207     panelArray.removeAll();
208     stepArea.setText("");
209
210     JLabelArray = new JLabel[array.length];
211     for (int i = 0; i < array.length; i++) {
212         JLabelArray[i] = new JLabel(String.valueOf(array[i]));
213         JLabelArray[i].setFont(new Font("Arial", Font.BOLD, 24));
214         JLabelArray[i].setOpaque(true);
215         JLabelArray[i].setBackground(Color.WHITE);
216         JLabelArray[i].setBorder(BorderFactory.createLineBorder(Color.BLACK));
217         JLabelArray[i].setPreferredSize(new Dimension(50, 50));
218         JLabelArray[i].setHorizontalAlignment(SwingConstants.CENTER);
219         panelArray.add(JLabelArray[i]);
220     }
221
222     generateMergeSteps(0, array.length - 1);
223
224     stepButton.setEnabled(true);
225     stepArea.setText("Array Awal: " + Arrays.toString(array) + "\n\n");
226
227     panelArray.revalidate();
228     panelArray.repaint();
229 }

```

33. Pada method **performStep**, secara bertahap melakukan penggabungan bagian-bagian daftar, dimulai dengan mengambil tugas penggabungan dari antrian, membandingkan elemen-elemen, menyalin ke daftar sementara, dan akhirnya menempatkan hasil gabungan kembali ke daftar utama


```

100 private void performStep() {
101     resetHighlights();
102
103     if (!isMerging && !mergeQueue.isEmpty()) {
104         int[] range = mergeQueue.poll();
105         left = range[0];
106         mid = range[1];
107         right = range[2];
108
109         temp = new int[right - left + 1];
110         i = left;
111         j = mid + 1;
112         k = 0;
113         copying = false;
114         isMerging = true;
115
116         logStep("Mulai merge range [" + left + ".." + right + "]");
117         highlightRange(left, right);
118         return;
119     }
120
121     if (isMerging && !copying) {
122         highlightRange(left, right);
123
124         if (i <= mid && j <= right) {
125             highlightCompare(i, j);
126             if (array[i] <= array[j]) {
127                 logStep("Bandingkan " + array[i] + " <= " + array[j] + ". Salin " + array[i] + " ke temp.");
128                 temp[k++] = array[i++];
129             } else {
130                 logStep("Bandingkan " + array[i] + " > " + array[j] + ". Salin " + array[j] + " ke temp.");
131                 temp[k++] = array[j++];
132             }
133             stepArea.append(" → Temp: " + Arrays.toString(Arrays.copyOf(temp, k)) + "\n\n");
134             return;
135         }
136
137         if (i <= mid) {
138             logStep("Salin sisa dari kiri: " + array[i]);
139             temp[k++] = array[i++];
140             stepArea.append(" → Temp: " + Arrays.toString(Arrays.copyOf(temp, k)) + "\n\n");
141             return;
142         }
143
144         if (j <= right) {
145             logStep("Salin sisa dari kanan: " + array[j]);
146             temp[k++] = array[j++];
147             stepArea.append(" → Temp: " + Arrays.toString(Arrays.copyOf(temp, k)) + "\n\n");
148             return;
149         }
150
151         copying = true;
152         k = 0;
153         logStep("Selesai mengisi temp. Mulai menyalin kembali ke array utama.");
154         stepArea.append(" → Temp Lengkap: " + Arrays.toString(temp) + "\n");
155         return;
156     }
157
158     if (isMerging && copying) {
159         if (k < temp.length) {
160             logStep("Salin " + temp[k] + " dari temp ke posisi " + (left + k));
161             array[left + k] = temp[k];
162             updateLabels();
163             highlightCopy(left + k);
164             k++;
165             return;
166         } else {
167             isMerging = false;
168             copying = false;
169             logStep("Selesai merge range [" + left + ".." + right + "]");
170
171             if (mergeQueue.isEmpty() && !isMerging) {
172                 stepArea.append("\n--- MERGE SORT SELESAI ---\n");
173                 stepArea.append("Hasil Akhir: " + Arrays.toString(array) + "\n");
174                 stepButton.setEnabled(false);
175                 JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
176                 for (JLabel label : labelArray)
177                     label.setBackground(Color.LIGHT_GRAY);
178             }
179         }
180     }
181 }

```

34. Pada method **resetHighlights**, mengatur ulang warna latar belakang semua label visual di layar menjadi putih, sehingga menghilangkan penyorotan dari langkah pengurutan sebelumnya

```

246 private void resetHighlights() {
247     if (labelArray == null) return;
248     for (JLabel label : labelArray) {
249         label.setBackground(Color.WHITE);
250     }
251 }

```

35. Pada method **reset**, mengembalikan seluruh program ke kondisi awal yang bersih dengan mengosongkan input, menghapus tampilan visual, membersihkan area log, menonaktifkan tombol, serta mengosongkan antrian penggabungan dan status pengurutan


```

268 private void reset() {
269     if (inputField != null) inputField.setText("");
270     if (panelArray != null) {
271         panelArray.removeAll();
272         panelArray.revalidate();
273         panelArray.repaint();
274     }
275     if (stepArea != null) stepArea.setText("");
276     if (stepButton != null) stepButton.setEnabled(false);
277     if (mergeQueue != null) mergeQueue.clear();
278     isMerging = false;
279     copying = false;
280     stepCount = 1;
281     array = null;
282     labelArray = null;
283 }

```

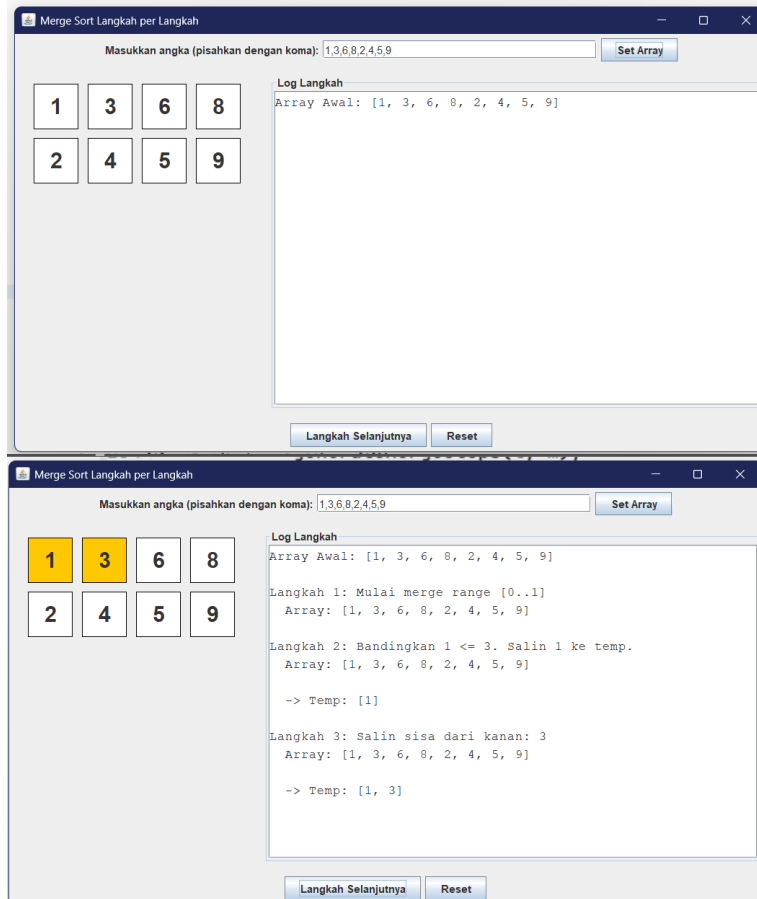
36. Pada method baru, yaitu **GenerateMergeSteps**, akan berjalan untuk mengisi antrian penggabungan dengan urutan bagian-bagian daftar yang harus digabungkan agar seluruh daftar dapat diurutkan

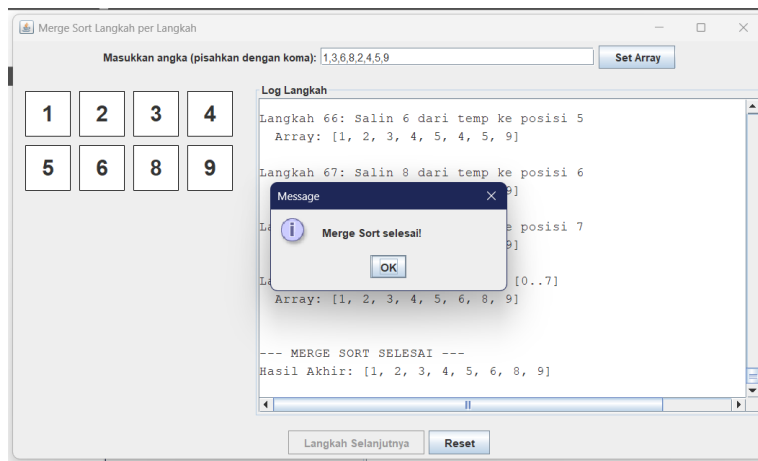
```

231 private void generateMergeSteps(int l, int r) {
232     if (l < r) {
233         int m = l + (r - l) / 2;
234         generateMergeSteps(l, m);
235         generateMergeSteps(m + 1, r);
236         mergeQueue.add(new int[] { l, m, r });
237     }
238 }

```

37. Berikut adalah hasilnya





D. Kesimpulan

Dari praktikum yang telah dilakukan, dapat diambil kesimpulan bahwa pemilihan algoritma sorting sangat bergantung pada kebutuhan dan tipe atau karakteristik data. Setiap metode pengurutan memiliki keunggulan dan keterbatasannya, sehingga pemahaman terhadap prinsip kerja masing-masing membantu dalam menentukan solusi yang paling efisien untuk aplikasi atau sistem yang nantinya akan dikembangkan.